

CarHunter — Complete Build Spec & Master Prompt

CarHunter documentation · source: CarHunter_Build_Spec.md · generated June 2026

CarHunter — Complete Build Spec & Master Prompt

One document. Hand it to any AI builder (Cursor, Claude Code, v0, Lovable, Bolt) or a developer. It contains the prompt, the architecture, the database schema, the scoring math, the scraper logic, the dashboard spec, and the setup steps. Build it top to bottom.

0. THE MASTER PROMPT (paste this first)

Build **CarHunter**, a self-running web app that finds undervalued private-party cars on **KSL Cars** and **Facebook Marketplace**, scores each one against a wholesale-profit buy-box, and surfaces only the deals worth pursuing. I'm a wholesale car buyer leaving the auctions — the biggest problem in my business is finding undervalued cars before anyone else, so that is the one job this app must do perfectly. It scrapes on a schedule, scores every listing, ranks the deals, alerts me on the best ones, and lets me work them from a pipeline board. I never touch a scraper or a database. **Fixed stack — do not substitute:** Convex (database, scheduled cron jobs, server functions, reactive queries), Daytona (sandboxed runtime for the scrapers), Playwright (scrapping), React + Vite + Tailwind (frontend). TypeScript everywhere except the Python scrapers. **Improve on three codebases I'm providing, do not start from zero:** (1) facebook-marketplace-scraper-1.1.0 — working FB Marketplace Playwright scraper with a web UI; reuse its scrape + parse + UI, drop its local SQLite. (2) scrape_marketplace.py — FB scraper with session cookies and a clean settings block; reuse the login/cookie + search-URL logic. (3) KSLHax-1.2 — KSL scraper with a deal-scoring method in assets/new_score_method.py; reuse its KSL parsing and evolve its score into the formula in §4. Merge all three into one multi-source engine backed by Convex and run inside Daytona sandboxes. Build everything in this document: the schema (§3), the scoring engine (§4), the scrapers (§5), the Convex backend (§6), the Daytona integration (§7), the dashboard (§8), and seed my buy-box (§2). Deliver a running app plus a README (§10). Make all the iterations yourself until it runs end-to-end — don't stop at a skeleton.

1. What the finished product does (definition of done)

- 1 I open the app. My buy-box searches are **already running on a schedule** — I set up nothing.
- 2 I see a **live, ranked feed** of private-party cars: photo, year/make/model, miles, asking price, estimated resale value, **estimated profit**, deal score, source, distance, days listed, and a link to the listing.
- 3 Anything above my margin threshold (default **\$1,500 profit/unit**) is flagged hot and **pushed to me** (email/SMS) the moment it's found.
- 4 I click **Pursue / Pass / Contacted** on each card. Pursued cars move to a **pipeline board** (Lead → Contacted → Negotiating → Bought → Flipped).
- 5 Price drops, relists, and "sold/gone" are tracked automatically. Deduped to **one row per real car** across both sources.
- 6 I never open a scraper, a terminal, or a database. The auctions are gone.

The product is the ranked deal feed. Everything else serves it.

2. My buy-box (seed these as starter saved searches)

These are the cars I actually move. Pre-load each as an active scheduled search around **Salt Lake City, UT (84104)**, 150-mile radius, running every 15 minutes.

Make/Model	Year band	Miles band	Notes
Chevrolet Traverse	2019–2024	30k–110k	LT/leather, AWD favored
VW Tiguan / Atlas	2019–2023	30k–120k	SE+, 3-row Atlas a plus
Mazda CX-5 / CX-9 / Mazda3	2018–2023	25k–120k	fast retail movers
GMC Terrain / Chevy Equinox	2017–2023	40k–130k	mainstream, easy out
Jeep Wrangler Unlimited	2018–2023	20k–110k	brings strong money
Ford Edge / Escape	2017–2023	40k–130k	Edge Sport/Titanium
Acura MDX	2016–2022	40k–120k	demand sleeper

Global buy-box defaults: price \$2,000–\$28,000 · mileage 25k–130k · year 2016–2024 · **clean title required** · AWD/4WD preferred (not required) · max days listed 30 · exclude salvage/rebuilt/branded unless I opt in per-search.

3. Database schema (Convex — [convex/schema.ts](#))

```
import { defineSchema, defineTable } from "convex/server";
import { v } from "convex/values";

export default defineSchema({
  // A saved buy-box that runs on a schedule
  searches: defineTable({
    name: v.string(),
    active: v.boolean(),
    sources: v.array(v.string()), // ["ksl", "facebook"]
    location: v.string(), // "Salt Lake City, UT"
    zip: v.string(),
    radiusMiles: v.number(),
    priceMin: v.number(),
    priceMax: v.number(),
    yearMin: v.number(),
    yearMax: v.number(),
    mileageMin: v.number(),
    mileageMax: v.number(),
    makes: v.array(v.string()),
    models: v.array(v.string()),
    maxDaysListed: v.number(),
    cleanTitleOnly: v.boolean(),
    intervalMinutes: v.number(), // how often to run
    lastRunAt: v.optional(v.number()),
    createdAt: v.number(),
  }).index("by_active", ["active"]),

  // One row per real car (deduped across sources)
  listings: defineTable({
    dedupeKey: v.string(), // see §5 dedupe
    source: v.string(), // "ksl" | "facebook"
    sourceListingId: v.string(),
    url: v.string(),
    title: v.string(),
    year: v.optional(v.number()),
    make: v.optional(v.string()),
    model: v.optional(v.string()),
  })
```

```

    trim: v.optional(v.string()),
    vin: v.optional(v.string()),
    mileage: v.optional(v.number()),
    price: v.number(),
    titleStatus: v.optional(v.string()), // "clean" | "salvage" | "rebuilt" | "unknown"
    location: v.optional(v.string()),
    distanceMiles: v.optional(v.number()),
    sellerType: v.optional(v.string()), // "private" | "dealer"
    photoUrl: v.optional(v.string()),
    photos: v.optional(v.array(v.string())),
    description: v.optional(v.string()),
    firstSeenAt: v.number(),
    lastSeenAt: v.number(),
    daysListed: v.optional(v.number()),
    priceHistory: v.array(v.object({ price: v.number(), at: v.number() })),
    status: v.string(), // "active" | "price_drop" | "gone" | "sold"
    // valuation + scoring (filled by scoring engine)
    estValue: v.optional(v.number()),
    estRecon: v.optional(v.number()),
    estFees: v.optional(v.number()),
    estProfit: v.optional(v.number()),
    dealScore: v.optional(v.number()), // 0-100
    hot: v.optional(v.boolean()),
    // workflow
    decision: v.optional(v.string()), // "new" | "pursue" | "pass" | "contacted"
    matchedSearchId: v.optional(v.id("searches")),
  })
  .index("by_dedupeKey", ["dedupeKey"])
  .index("by_score", ["dealScore"])
  .index("by_decision", ["decision"])
  .index("by_status", ["status"]),

// Comp set used to value a car (cached MarketCheck-style retail comps)
comps: defineTable({
  ymm: v.string(), // "2023|Chevrolet|Traverse"
  mileageBucket: v.string(), // "70k-90k"
  sampleSize: v.number(),
  medianRetail: v.number(),
  p25Retail: v.number(),
  p75Retail: v.number(),
  refreshedAt: v.number(),
}).index("by_ymm_bucket", ["ymm", "mileageBucket"]),

// Pipeline board
pipeline: defineTable({
  listingId: v.id("listings"),
  stage: v.string(), //
"lead"|"contacted"|"negotiating"|"bought"|"flipped"|"dead"
  targetBuy: v.optional(v.number()),
  walkAway: v.optional(v.number()),
  notes: v.optional(v.string()),
  updatedAt: v.number(),
}).index("by_stage", ["stage"]),

// Alert log
alerts: defineTable({
  listingId: v.id("listings"),
  channel: v.string(), // "email"|"sms"
  sentAt: v.number(),
  score: v.number(),
}),

// Single-row app settings

```

```

settings: defineTable({
  marginThreshold: v.number(),          // default 1500
  alertEmail: v.optional(v.string()),
  alertPhone: v.optional(v.string()),
  fbSessionCookie: v.optional(v.string()),
  daytonaApiKey: v.optional(v.string()),
  feesFlat: v.number(),                  // default 400
}),
});

```

4. The scoring engine (the heart of the app)

Server function in `convex/scoring.ts`. Runs on every newly inserted/updated listing.

Step 1 — Estimate resale value (`estValue`).

- Look up cached comps in the `comps` table by `ymm` + mileage bucket. Use `medianRetail` as the anchor.
- If no comp exists, call the valuation source (MarketCheck API if a key is set; otherwise fall back to a built-in depreciation curve seeded from my buy-box). Cache the result into `comps` for 7 days.
- Adjust for mileage delta vs the bucket median ($\pm \$0.06/\text{mile}$), title status (salvage/rebuilt = $\times 0.65$), and AWD/4WD (+3%).

Step 2 — Estimate recon (`estRecon`). Default \$400. Bump from description keywords: "salvage/rebuilt" +\$2,500, "doesn't start/won't start/dead/mechanic special" +\$2,000, "needs work/as-is" +\$1,000, "accident/damage" +\$800, "check engine" +\$600, "new tires/brakes" −\$200.

Step 3 — Fees (`estFees`). Flat `settings.feesFlat` (default \$400): title/transport/detail.

Step 4 — Profit + score.

```
estProfit = estValue - price - estRecon - estFees
```

```

dealScore (0-100) =
  45 * clamp(estProfit / 4000, 0, 1)          // profit is king
+ 20 * clamp((estValue - price) / estValue, 0, 1) // margin %
+ 15 * freshnessBonus(daysListed)             // newer listing = more bonus
+ 10 * mileageFit(mileage, buyboxBand)
+ 10 * titleBonus(titleStatus)                // clean = full 10

```

`hot = estProfit >= settings.marginThreshold` (default \$1,500).

Step 5 — Suggested numbers (write to pipeline when pursued):

```

targetBuy = estValue - estRecon - estFees - marginThreshold
walkAway  = estValue - estRecon - estFees - (marginThreshold * 0.6)

```

Reuse `KSLHax-1.2/assets/new_score_method.py` as the starting point for Step 1–4, then upgrade it to comp-based valuation as above.

5. The scrapers (merged, runs in Daytona)

Two Python Playwright scrapers in `scrapers/`, plus a shared parser. Each scraper takes a search config (JSON), returns a list of normalized listing dicts, and posts them back to Convex via an HTTP action.

`scrapers/facebook.py` — base it on `facebook-marketplace-scraper-1.1.0` + `scrape_marketplace.py`. Inject the FB session cookie from Convex `settings.fbSessionCookie` so we never store a password. Build the search URL from the config (location, price, mileage, year, make, model, days listed). Block images/media for speed (the

`abort_requests` pattern in `scrape_marketplace.py`). Scroll N times, parse cards.

`scrapers/ksl.py` — base it on `KSLHax-1.2`. KSL Cars has a clean query API/URL structure; parse listing tiles directly.

`scrapers/parse.py` — shared normalizer. Extract `year/make/model/trim` from the title (regex + a make/model dictionary — reuse `dist/web/js/car-brands.js` from the FB repo), mileage, price, title status keywords, seller type (filter out dealers — **private party only**), photos, location → distance from zip.

Dedupe key: `vin` if present; else `sha1(year|make|model|round(mileage, -3)|zip3)`. On post-back, Convex upserts by `dedupeKey`: new car = insert + score; seen car with lower price = append to `priceHistory`, set `status:"price_drop"`, rescore; not seen this run for 48h = `status:"gone"`.

Normalized listing shape returned by every scraper:

```
{
  "source": "ksl",
  "sourceListingId": "abc123",
  "url": "https://cars.ksl.com/listing/abc123",
  "title": "2021 Chevrolet Traverse LT AWD",
  "price": 19500, "mileage": 78000,
  "year": 2021, "make": "Chevrolet", "model": "Traverse", "trim": "LT",
  "vin": null, "titleStatus": "clean", "sellerType": "private",
  "location": "Sandy, UT", "photoUrl": "...", "photos": ["..."],
  "description": "..."
}
```

6. Convex backend (`convex/`)

- `searches.ts` — CRUD for buy-boxes. `ListActive` query for the cron.
- `listings.ts` — `upsertFromScrape` (mutation: dedupe + price history + status), `feed` (query: ranked, filterable), `setDecision` (pursue/pass/contacted).
- `scoring.ts` — `scoreListing` (internal mutation per §4), `getOrFetchComp` (action with `MarketCheck` fallback + 7-day cache).
- `pipeline.ts` — move stage, set target/walk-away/notes.
- `alerts.ts` — `sendHotAlert` action (Resend for email, Twilio for SMS); dedupe so a car only alerts once unless it drops price.
- `crons.ts` — every minute, find `searches` whose `lastRunAt` + `intervalMinutes` is due, and for each call the `Daytona runner` action (§7). Also a daily job to mark stale listings `gone` and refresh comps.
- `http.ts` — `POST /ingest` endpoint the Daytona sandbox calls to push scraped listings back (auth via a shared secret).

7. Daytona integration (`convex/daytona.ts`)

Action `runSearchInSandbox(searchId)`:

- 1 Read the search config + FB cookie + ingest secret from Convex.
- 2 Create a Daytona sandbox from a prebuilt image that has Python + Playwright + the `scrapers/` folder.
- 3 Run `python run.py --config <json>` inside the sandbox; the scraper posts results to Convex `/ingest` as it goes.
- 4 Tear the sandbox down. Update `search.lastRunAt`.

- 5 Run searches in parallel (one sandbox each), capped at a concurrency limit. Log failures; never let one bad run block others.

`scrapers/run.py` is the sandbox entrypoint: picks `ksl/facebook` per `sources`, runs them, normalizes, batches POSTs to `/ingest`.

Why Daytona: scraping is isolated per run, scales horizontally, keeps the session/IP separate from the app, and disappears when done. No scraper ever runs on the app server.

8. The dashboard (web/ — React + Vite + Tailwind)

Reactive via Convex `useQuery`. Five views, mobile-friendly (I work it from the lot).

- 1 **Deal Feed (home)**. Ranked cards, hottest first. Each card: photo, YMM + trim, miles, **asking price** big, **est. value** and **est. profit** (green), score chip (color by tier), source badge, distance, days listed, "View listing" link, and three buttons: **Pursue / Pass / Contacted**. Top filter bar: source, make, price, min profit, min score, max days listed, sort. A red **HOT** ribbon when `hot`.
- 2 **Pipeline board**. Kanban: Lead → Contacted → Negotiating → Bought → Flipped → Dead. Drag cards between stages. Each shows target-buy / walk-away and a notes field.
- 3 **Search Builder**. Form to create/edit a buy-box (all §3 `searches` fields). Toggle active. Shows last-run time and # new deals found.
- 4 **Settings**. Margin threshold, flat fees, alert email/phone, FB session cookie paste, Daytona API key, MarketCheck key.
- 5 **Car detail drawer**. Full photos, description, price history chart, comp set used, the profit math broken out line by line, and the suggested target/walk-away.

Visual language: clean, dense, dealer-tool feel. Green = money, red = pass/risk, amber = verify. Same logic as my auction field notes.

9. Build order (do these in sequence, verify each runs)

- 1 `convex init`, drop in `schema.ts`, push. Seed `settings` (margin 1500, fees 400) and the §2 buy-box searches.
- 2 Build `scrapers/parse.py` + a local CLI test against saved HTML so parsing is solid before any network.
- 3 `facebook.py` and `ksl.py`; run locally, confirm normalized output.
- 4 Convex `listings.upsertFromScrape` + `http.ts /ingest`; wire scrapers to POST. Confirm cars land + dedupe.
- 5 `scoring.ts` + comp fetch/fallback; confirm `estProfit`, `dealScore`, `hot` populate.
- 6 Daytona runner + `crons.ts`; confirm searches fire on schedule into sandboxes.
- 7 Dashboard: feed → search builder → pipeline → settings → detail drawer.
- 8 Alerts (Resend/Twilio). 9. Polish, mobile pass, README.

10. README contents to deliver

- One-paragraph what-it-is.
- Prereqs: Node 20+, Python 3.11+, Convex account, Daytona account, (optional) MarketCheck/Resend/Twilio keys.

- Setup: `npm i, npx convex dev`, set env vars (`CONVEX_DEPLOYMENT`, `DAYTONA_API_KEY`, `INGEST_SECRET`, `MARKETCHECK_KEY`, `RESEND_KEY`, `TWILIO_*`), build the Daytona scraper image, paste FB session cookie in Settings.
 - How to add a buy-box, read the feed, work the pipeline.
 - The scoring formula in plain English (§4) so I can tune it.
-

11. Honest constraints (so the build doesn't surprise me)

- **KSL and Facebook both restrict automated scraping** in their terms; FB actively blocks bots. The session-cookie + Daytona-sandbox approach is the practical workaround, but expect to refresh the FB cookie periodically and rotate if blocked. Build ret/backoff + a "session expired" banner in Settings.
- **Valuation is only as good as the comps.** Wire MarketCheck if I have a key; the built-in curve is a fallback, not gospel. Every car shows its comp set so I can sanity-check.
- **Private-party only:** hard-filter out dealer listings — that's where the margin is.
- This is a tool that finds and ranks; **I make the final call and the contact.** It does not message sellers or move money automatically.

End of spec. Build all of it, in order, until the app runs end-to-end.